

Figure 1: Deployment of Web bundles in OSGi runtime

On the other hand, *org.eclipse.equinox.http.jetty* is a multi-bundle Jetty-based implementation, which provides full support for the Servlet 2.4 specification. It is made up of the following bundles:

- *org.eclipse.equinox.http.jetty*
- *org.eclipse.equinox.http.servlet*
- *org.mortbay.jetty*
- *org.apache.commons.logging*
- *javax.servlet*
- *org.eclipse.equinox.http.registry*

Development and deployment of Web applications in Equinox is different from traditional Web applications. Though this is an application server capable of running servlets and serving HTML files, it cannot understand what a WAR file is. In an OSGi environment, what makes sense is a bundle and not a WAR—so Web applications should be packaged as a bundle. This bundle should extend HTTP service-specific extension points (*org.eclipse.equinox.http.registry.resources* and *org.eclipse.equinox.http.registry.servlets*) to be able to act as a Web application. The first extension point is used to specify the static files to be served by this Web application bundle, while the second is used to specify the servlets the container should run for this Web application bundle.

Embedding Equinox in an existing application container

The unit of deployment in this methodology is a Web Application Archive (WAR) file. The bridge servlet inside the WAR acts as a front controller, by redirecting incoming HTTP requests to the correct OSGi bundle. It provides controls to manage the embedded framework. It reads the path information in the incoming request and decides where to redirect. If the incoming request is for controlling the framework, then it starts or stops the framework based on the request. If the request is for some other HTML file or servlet in the OSGi Web app, it redirects the request to the appropriate resource.

Just as in the case of embedding HTTP server in Equinox, Web OSGi bundles extending HTTP specific extension points are packaged inside WARs and deployed to the application server. If the Web application needs to serve JSP, then the following Jasper OSGi bundles need to be deployed in the

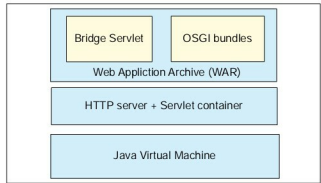


Figure 2: Deployment of Web bundles in application server

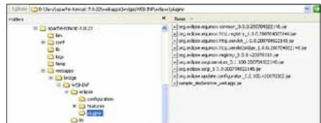


Figure 3: Packaging Web application bundles in bridge.war file

container:

- *org.apache.jasper_5.5.17.v200806031609.jar*
- *org.eclipse.equinox.jsp.jasper_1.0.100.v20080427-0830.jar*
- *org.eclipse.equinox.jsp.jasper.registry_1.0.0.v20080427-0830.jar*

As a best practice, instead of deploying multiple OSGi bundles, all the bundles are placed inside *bridge.war*.

A sample WAR package structure is shown in Figure 3.

Testing OSGi applications

Unit testing of OSGi bundles can be done by writing JUnit test cases. Mock object frameworks like Mockito can be used to create mock objects in JUnit test cases. Dependency injection frameworks like Blueprint can be used to inject objects. Even container objects like *BundleContext* can be injected to simulate the container environment. Though this form of testing can uncover some defects, the absence of a real container in all these tests will allow many defects through. A well-accepted solution to this problem is a tool from the Pax Runner family called Pax Exam. Pax Exam wraps the test cases as an OSGi service bundle, and deploys the bundle in the OSGi runtime using Pax Runner. Pax Exam invokes the tests and records the results. Pax Exam is driven fully by *annotations*, which are used to specify the tests to run, bundles to deploy, dependencies to inject, etc. Tycho, described earlier, can also be used for testing OSGi bundles. 

By: AnandaVelMurugan Chandra Mohan

The author is a Java Web application developer. He likes to explore new technologies and build business cases around them.